

DilutionAttention: a parameter-free attention operator that divides each bid by the demand its key has already absorbed

Technical report, version 0.1

Nick Royer

Royer Research Labs, LLC

<https://github.com/Royer-Research-Labs/Dilution-Attention>

September 2026

Abstract

We describe *dilution attention*, a causal attention operator with no learnable parameters. Each query’s softmax bid on a key is divided by the cumulative demand that key has already received from earlier queries, and the resulting shares are renormalised. The operator is an exact reformulation of inclusive temporal attention applied to row-normalised scores, used here as the attention of a causal Transformer language model. At 209M parameters, trained at a 16K-token context on a Chinchilla-matched budget with *no position encoding*, dilution models retrieve planted keywords at eight times their training context in both seeds, where softmax models trained identically reach 0.17–0.19 at their own context and chance beyond it. On long books, dilution’s loss stays flat to eight times its training context while every softmax model’s rises steeply, and past the trained context every softmax model we tested writes incoherent text while dilution’s samples keep their in-context quality. Inside the trained context, dilution’s loss and judged sample quality track the stronger softmax-with-RoPE control on in-domain text (level at 16K, behind it at 1K), but on out-of-domain books it trails that control by 0.05–0.24 nats. We release fused Triton kernels with exact gradients, the training and evaluation code, run records for more than 200 training runs, model checkpoints and an evidence ledger, and we state the limits: a 209M scale, a synthetic retrieval test, and training about $2.2\times$ slower than softmax at 16K.

1 Introduction

Softmax attention distributes a fixed budget of weight over the visible keys, and nothing records how much of that budget other queries have already spent on a key. Trained models use this freedom in recognisable ways: they park unneeded mass on a few positions, usually the first token (the *attention sink* [9, 10]), and they depend on position encodings whose behaviour outside the trained range limits length generalisation [12, 13, 14, 15].

Dilution attention adds a price. A query still places softmax bids, but each bid is divided by the demand its key has accumulated from earlier queries, so a key that many queries have already claimed yields a smaller share. The idea has a close precedent: temporal attention in recurrent encoder–decoders divides the current alignment by the alignments of earlier decoder steps [1, 2]. Our contribution is not that division; it is (i) an inclusive, row-calibrated form that is well defined in causal self-attention, where the exclusive form degenerates; (ii) fused kernels with exact gradients that make it trainable at 16K–256K tokens; and (iii) a controlled empirical study, with matched

softmax baselines and position-encoding controls, of what it does to loss, retrieval, generation and attention structure.

The central empirical result is length extrapolation without position encoding (Figure 1). The comparison that isolates the operator holds everything else fixed: the same architecture, data, schedule and learning rate, and no position encoding on either side. We report every seed, including the ones that make the story less tidy: extrapolation range varies by seed even when loss replicates to three decimals.

2 The operator

For causal logits $z_{ij} = q_i \cdot k_j / \sqrt{d}$, $j \leq i$:

$$p_{ij} = \frac{\exp z_{ij}}{\sum_{k \leq i} \exp z_{ik}} \quad \text{softmax bids} \quad (1)$$

$$d_{ij} = \sum_{r \leq i} p_{rj} \quad \text{demand on key } j \text{ (inclusive)} \quad (2)$$

$$s_{ij} = \frac{p_{ij}}{d_{ij} + \epsilon}, \quad A_{ij} = \frac{s_{ij}}{\sum_{k \leq i} s_{ik}}, \quad o_i = \sum_{j \leq i} A_{ij} v_j. \quad (3)$$

The operator has no parameters, is causal, is invariant to shifting a row’s logits, and keeps every pre-normalisation share in $[0, 1]$ (the diagonal share is exactly $p_{ii}/(0 + p_{ii}) = 1$ up to ϵ). It is $O(T^2)$ like softmax attention. Cached decoding needs the KV cache plus one fp32 demand accumulator per key and head.

Relation to temporal attention. Writing $z'_{ij} = \log p_{ij}$, the share is $s_{ij} = \exp z'_{ij} / \sum_{r \leq i} \exp z'_{rj}$: dilution is inclusive temporal attention on row-calibrated logits. Temporal attention [1, 2] divides by *exclusive* history, $\sum_{r < i}$. In causal self-attention that denominator is exactly zero at each key’s first query, which is the key itself, so the diagonal wins every row by $1/\epsilon$ and attention collapses to the identity at initialisation. A model trained with exclusive history (and row-normalised bids) escapes the collapse in every layer but the first, which stays an identity map. Against inclusive dilution trained with the same recipe it loses 0.072 nats, and retrieval at 7.9K tokens falls from 1.00 to 0.62, no better than a control with no demand step at all (64M, one seed each; S18). The raw-exponential cells of the temporal-attention factorial remain untested: our fp32 implementation overflowed, and a numerically stable cumulative-logsumexp form has not been built.

The bid must be a softmax. The share’s gradient scales as $1/p$. Of the bid functions tried, only softmax and sigmoid train; sigmoid is 0.33 nats worse. Bids whose Jacobian lacks a factor of the bid (ReLU, ReLU², a min-shifted linear bid) diverge, as that predicts. Softplus also diverged although its Jacobian does carry the factor; that failure is measured, not explained. With softmax bids the demand step is worth 0.178 nats and lifts retrieval at 7.9K from 0.59 to 1.00 (64M, learning-rate matched; S3, S4).

3 Implementation

The column prefix sum is the operator’s only sequential coupling, so standard fused-attention kernels cannot express it as a score modifier. Our Triton kernels compute online softmax statistics, then a key-parallel pass that carries demand in registers and saves checkpoints every 32 rows; the

arm (209M, 16K, 1×)	val. loss (two seeds)	NIAH at 16K	retrieval beyond 16K	tok/s
alt6 (RoPE softmax + dilution)	3.0385 / 3.0399	1.00 / 1.00	3–4× with NTK	44.0K
dilution, no position encoding	3.0569 / 3.0548	1.00 / 1.00	8× in both seeds	33.4K
softmax + RoPE	3.0883 / 3.0787	0.79 / 0.64	1.5× in one seed	71.9K
softmax, no position encoding	3.2118 / 3.2907*	0.19 / 0.17	chance	74.7K

Table 1: The 209M comparison. *This seed hit a burst of gradient spikes and never recovered the loss. NIAH chance is 0.14.

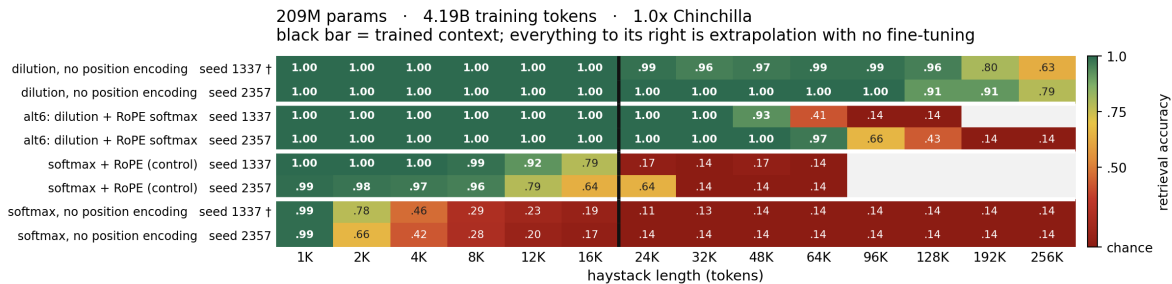
backward walks row blocks in reverse, so the per-key demand suffix is a running sum, and applies the softmax-Jacobian correction in a separate pass. Gradients are exact; a variant that treated demand as a constant in the backward was 23% faster and trained an unstable objective (S26). Forward plus backward takes 5.46ms at batch 8, 8 heads, 4K tokens and 30.2ms at the 209M training shape (2 sequences, 12 heads, 16K tokens) on an RTX PRO 6000; a 256K-token forward runs in 17.9 GiB. Softmax attention (SDPA) trains 1.3× faster than dilution at 1K context and 2.2× faster at 16K. A full-schedule replay reproduced the loss of the first working kernels to 0.0007 nats.

4 Experimental setup

Models. Decoder-only Transformers with SwiGLU MLPs, no biases, tied embeddings and the GPT-2 vocabulary: a 64M family (12 layers, width 512, 8 heads) and a 209M family (24 layers, width 768, 12 heads). Arms differ only in the attention operator and position encoding: all-dilution with no position encoding; all-softmax with RoPE (base 5×10^5) or with none; *alt6*, which alternates RoPE-softmax layers with unrotated dilution layers; and, at 64M, dilution with RoPE on alternate layers. Dilution adds no parameters, so matched arms have identical parameter counts.

Training. FineWeb-Edu [19] (a 40B-token GPT-2-tokenised sample), 65,536 tokens per step at 209M, learning rate 6×10^{-4} , cosine over a 2×-Chinchilla [18] schedule of 8.4B tokens. “1×” is the step-64,000 checkpoint (4.19B tokens; learning rate 55.7% of peak); “2×” is the end of the schedule. The 209M models are trained at 16K and at 1K context, two seeds per arm; the 64M reliability studies use three seeds.

Evaluation. *Loss*: one common validation slice per scale, because in-run windows differ with batch size. *Retrieval (NIAH)*: a synthetic, likelihood-scored test; a fixed 54-token paragraph is repeated to length, the sentence “The secret keyword is *w*.” is planted at evenly spaced depths, and the model’s likelihood is compared across 8 fixed single-token keywords, of which the one the model prefers on a needle-free prompt is disqualified, so chance is 1/7. It measures copying one planted token out of repetitive filler, not reasoning. Beyond the trained context, RoPE layers use dynamic NTK base scaling [15]. *Long-document perplexity*: loss by position on PG-19 books [20] (Section 5.2). *Generation*: 256-token continuations of validation prompts, scored in bits per byte by a larger judge model, SmolLM2-1.7B [21], which sees only the last 4,096 tokens of each prompt; paired over prompts and read together with repetition [22, 23].



24-layer d768 models trained at 16K on the same corpus, schedule and learning rate; rows differ only in the attention operator, its position encoding and the seed. Softmax without position encoding seed 2357 hit gradient spikes early in training (docs/signals.md 524). Likelihood-scored keyword retrieval, 7 scored needles, chance 1/7: 20 depth placements in context, 10 beyond. RoPE rows use dynamic NTK base scaling beyond the trained context. † Scored with the earlier niah-v3 prompts (question appended past the stated length; control prior measured on a prompt capped at 5,400 tokens). These checkpoints were trained on past this step and cannot be re-scored; every other v3 result in these figures was re-scored under v4 (docs/research-results.md, NIAH protocol versions).

Figure 1: NIAH retrieval past the training context, 209M parameters, 16K training context, 1× Chinchilla, every seed. Cells right of the black bar are extrapolation with no fine-tuning. Rows marked † were scored with an earlier protocol version whose checkpoints no longer exist at this step; all other results were re-scored with the current one.

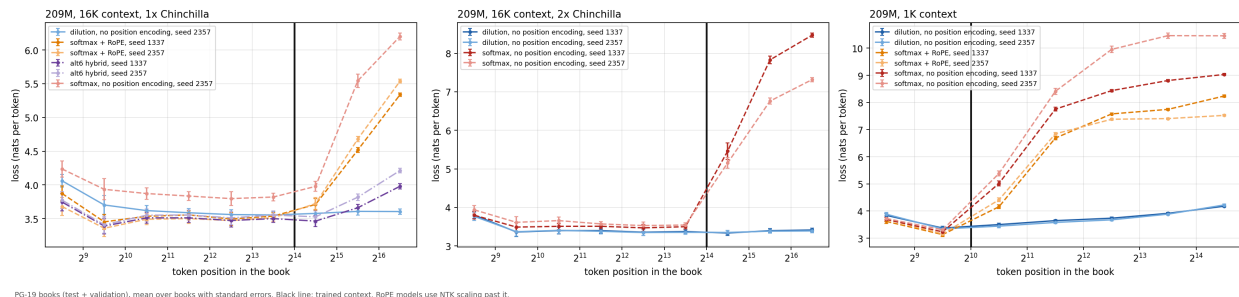


Figure 2: Loss by position on PG-19 books (29 books of at least 128K tokens for the 16K models, 120 of at least 32K for the 1K models), mean over books with standard errors. Black line: trained context. RoPE models use NTK scaling past it.

5 Results

5.1 Length extrapolation comes from the operator

With no position encoding on either side, dilution retrieves 1.00 across its 16K context and 0.91–1.00 out to 128K in both seeds, then degrades gradually (0.80–0.91 at 12×, 0.63–0.79 at 16×); softmax reaches 0.17–0.19 at its own trained context and chance past it (Figure 1, Table 1; S17, S24). Dilution without position encoding also beats softmax *with* RoPE on loss. No tested softmax configuration sustains strong retrieval far beyond its training context: the 16K-trained controls are at chance by 2× (one softmax + RoPE seed holds 0.64 at 1.5× with NTK scaling), and the 1K-trained controls keep partial retrieval near 2× (0.56 / 0.39 without position encoding) before degrading. The RoPE-bearing dilution and hybrid configurations also degrade eventually; NTK scaling extends some of them but does not remove the degradation.

5.2 Long-document perplexity

Retrieval is a synthetic test; loss by position on long real documents is the standard measurement of length extrapolation. Each model reads the first 128K (16K models) or 32K tokens (1K models)

209M models (nucleus sampling)	inside the trained context	2× past it	4× past it
16K, 1×: dilution, no position encoding	1.09–1.13	1.14	
softmax + RoPE	1.09–1.15	1.87 / 2.15	
alt6	1.06–1.10	1.20 / 1.20	
16K, 2×: dilution, no position encoding	1.00–1.04	1.00 / 1.02	
softmax, no position encoding	1.05–1.14	3.33 / 2.31	
1K: dilution, no position encoding	1.07–1.10	1.12 / 1.11	1.16 / 1.13
softmax + RoPE	1.05–1.09	2.45 / 2.58	2.97 / 2.62
softmax, no position encoding	1.07–1.10	2.26 / 2.37	2.25 / 2.77
human continuation	0.65–0.70		

Table 2: Judge bits per byte of 256-token continuations (lower is more natural; pairs are two seeds). 128–256 prompts per cell.

of long PG-19 books [20] teacher-forced, and the loss at every position is averaged within doubling position ranges (Figure 2; S27). Past the trained context the 16K dilution models stay flat to 8×, within 0.05 nats of their in-context loss. Paired over books, softmax + RoPE with NTK scaling is 0.9–1.1 nats worse than dilution at 4× and 1.7–1.9 at 8× (at 2× the difference, 0.13–0.15, is within noise), and softmax without position encoding is 2.6–5.1 nats worse at 8×. alt6 matches dilution at 2× and degrades from 4×. The 1K dilution models degrade gradually (+0.1 nats at 2×, +0.85 at 32×), while every 1K softmax model rises by 1.0–2.1 nats at 2× and 3.5–5.1 by 4×.

Inside the trained context the comparison differs from the in-domain validation slice. On these books, which are far from the training data, dilution trails softmax + RoPE by 0.05–0.08 nats at 16K and by 0.18–0.24 at 1K, and trails softmax without position encoding by 0.09–0.15 at 1K, most at the start of each book; on FineWeb-Edu validation text the same checkpoints put dilution 0.02–0.03 ahead of softmax + RoPE at 16K and 0.036 behind it at 1K. We have not tested why.

5.3 Extrapolation range varies by seed

Dilution’s loss replicates to 0.002–0.004 nats in every cell measured, but its reach does not. Trained on to 2× Chinchilla, both dilution seeds reach 2.857 / 2.854 and keep in-context retrieval, but one then holds only to about 2× (0.54 at 64K) while the other stays near-flat (0.97 at 64K, 0.83 at 128K). At 1K context the two seeds reach 4× and 8×. At 64M without position encoding, in-context retrieval itself forms in only about two seeds of three, at identical loss; the failed seeds never grow the strong retrieval head (53–76% of one head’s attention on the answer token in layers 6–9) that healthy seeds have. RoPE on alternate layers makes retrieval form in all three seeds (S22–S24). Extrapolation should be reported as a range over seeds.

5.4 Loss depends on context length

Once each operator has its best position mode, softmax + RoPE is ahead by 0.036 nats at 1K (209M, two seeds), the two are level within 0.007 at 8K (64M), and dilution leads at 16K: by 0.028 without position encoding and 0.044 for alt6 at 209M, and by 0.077–0.092 at 64M over three seeds. Earlier comparisons against learned-position softmax reported 0.19–0.43 nats and mostly measured that baseline’s failure (S1, S5–S9, S11, S21–S24). These margins are on in-domain validation text; on out-of-domain books (Section 5.2) dilution trails inside the trained context.

5.5 Generated text

Inside the trained context, judged sample quality tracks loss (Table 2; S25): dilution is level with softmax + RoPE at 16K, 0.03–0.12 bits per byte better than softmax without position encoding at $2\times$ Chinchilla, and about 0.01 behind softmax + RoPE at 1K, where softmax has the better loss. The test resolves loss gaps of about 0.2 nats (training the same dilution seed from $1\times$ to $2\times$ improves its samples by 0.05–0.11), not a few hundredths. Past the trained context, all the softmax models tested write incoherent text at $2\text{--}4\times$, and scored teacher-forced they assign the human continuation 6–9 nats per token; dilution’s samples stay within 0.06 of their in-context quality and its loss on the human text within 0.04. Because the judge sees only the last 4,096 tokens, this shows that dilution remains a working language model past its trained window, not that it uses information from far back. Greedy decoding loops in every model at this scale.

5.6 Attention structure

In three sampled layers of one 1,024-token window per model, averaged over heads, dilution strongly suppresses the token-0 sink: no sampled dilution layer puts more than 0.028 of its attention there, against 0.14–0.75 for 209M softmax without position encoding and 0.25–0.32 with RoPE. At depth its load is less concentrated across keys at similar per-query selectivity. Neither statistic tracks extrapolation range: two dilution seeds that differ in reach have the same sink and key concentration (S20). These are snapshots of trained models, not a causal account.

6 Related work

Temporal attention and coverage. Temporal attention [1, 2] is the closest equation-level precedent (Section 2); coverage models [4, 5, 3] feed accumulated attention back into a learned score or loss rather than dividing by it, and See et al. found post-hoc temporal division harmful in their summariser. *Balanced and competitive attention.* Sinkformers [6] balance rows and columns iteratively; Flowformer [7] and Slot Attention [8] create competition through conservation or normalisation over slots; none divides a causal softmax bid by prior demand. *Sinks and caches.* StreamingLLM and StableMask [9, 10] characterise and manage the attention sink; H2O [11] uses accumulated attention to evict cache entries. *Position encoding and length generalisation.* RoPE [12], NoPE [13], position interpolation [14] and YaRN/NTK scaling [15] target extrapolation through the position signal; our result is an operator-level effect measured with position encoding held fixed. *Kernels.* FlashAttention [16] and FlexAttention [17] fuse row-local softmax attention; dilution’s column prefix sum needs a different decomposition.

7 Limitations

- Scale: 64M and 209M parameters, one tokenizer and one corpus family. Whether the effects hold at billions of parameters is untested.
- Retrieval is measured with a synthetic, likelihood-scored keyword test. Generation is tested for local fluency only. Long-range use of information in free generation, retrieval by generation and long-context reasoning benchmarks are untested.
- Seeds: two per arm at 209M, three at 64M. Extrapolation range is seed-dependent.
- Cost: dilution trains $1.3\times$ (1K) to $2.2\times$ (16K) slower than SDPA softmax; the heaviest backward pass is latency-bound and further gains need a lower-level implementation.

- In-context loss out of domain: on long PG-19 books dilution trails softmax + RoPE inside the trained context by 0.05–0.08 nats at 16K and 0.18–0.24 at 1K, although it is level or ahead on in-domain text.
- Mechanism: why dilution extrapolates is not established causally.

8 Reproducibility

The release contains the operator’s fp32 reference and fused kernels with parity tests; training and evaluation code; the configuration, metrics and data provenance of every published run (200+); every result file behind the tables and figures, including per-sample sampling metrics from which a script recomputes every published statistic; model checkpoints; and an evidence ledger that records each finding with its runs, including corrections and withdrawn claims. Every number in this report traces to a run record and a result file in the repository.

References

- [1] B. Sankaran, H. Mi, Y. Al-Onaizan, A. Ittycheriah. Temporal attention model for neural machine translation. arXiv:1608.02927, 2016.
- [2] R. Paulus, C. Xiong, R. Socher. A deep reinforced model for abstractive summarization. ICLR 2018.
- [3] A. See, P. J. Liu, C. D. Manning. Get to the point: summarization with pointer-generator networks. ACL 2017.
- [4] Z. Tu, Z. Lu, Y. Liu, X. Liu, H. Li. Modeling coverage for neural machine translation. ACL 2016.
- [5] H. Mi, B. Sankaran, Z. Wang, A. Ittycheriah. Coverage embedding models for neural machine translation. EMNLP 2016.
- [6] M. E. Sander, P. Ablin, M. Blondel, G. Peyré. Sinkformers: Transformers with doubly stochastic attention. AISTATS 2022.
- [7] H. Wu, J. Wu, J. Xu, J. Wang, M. Long. Flowformer: linearizing Transformers with conservation flows. ICML 2022.
- [8] F. Locatello et al. Object-centric learning with slot attention. NeurIPS 2020.
- [9] G. Xiao, Y. Tian, B. Chen, S. Han, M. Lewis. Efficient streaming language models with attention sinks. ICLR 2024.
- [10] Q. Yin et al. StableMask: refining causal masking in decoder-only Transformer. ICML 2024.
- [11] Z. Zhang et al. H₂O: heavy-hitter oracle for efficient generative inference of large language models. NeurIPS 2023.
- [12] J. Su, Y. Lu, S. Pan, A. Murtadha, B. Wen, Y. Liu. RoFormer: enhanced Transformer with rotary position embedding. arXiv:2104.09864, 2021.
- [13] A. Kazemnejad, I. Padhi, K. Natesan Ramamurthy, P. Das, S. Reddy. The impact of positional encoding on length generalization in Transformers. NeurIPS 2023.
- [14] S. Chen, S. Wong, L. Chen, Y. Tian. Extending context window of large language models via positional interpolation. arXiv:2306.15595, 2023.

- [15] B. Peng, J. Quesnelle, H. Fan, E. Shippole. YaRN: efficient context window extension of large language models. ICLR 2024.
- [16] T. Dao, D. Y. Fu, S. Ermon, A. Rudra, C. Ré. FlashAttention: fast and memory-efficient exact attention with IO-awareness. NeurIPS 2022.
- [17] J. Dong, B. Feng, D. Guessous, Y. Liang, H. He. FlexAttention: a programming model for generating fused attention variants. MLSys 2025.
- [18] J. Hoffmann et al. Training compute-optimal large language models. NeurIPS 2022.
- [19] G. Penedo et al. The FineWeb datasets: decanting the web for the finest text data at scale. NeurIPS 2024 Datasets and Benchmarks.
- [20] J. W. Rae et al. Compressive Transformers for long-range sequence modelling. ICLR 2020.
- [21] L. B. Allal et al. SmolLM2: when smol goes big, data-centric training of a small language model. arXiv:2502.02737, 2025.
- [22] S. Welleck, I. Kulikov, S. Roller, E. Dinan, K. Cho, J. Weston. Neural text generation with unlikelihood training. ICLR 2020.
- [23] A. Holtzman, J. Buys, L. Du, M. Forbes, Y. Choi. The curious case of neural text degeneration. ICLR 2020.